



DOCUMENT À ENVOYER AU STAGIAIRE

Mission stagiaire DevOps — Phase 1

Comprendre, documenter et préparer l'environnement local & SaaS

Comprendre le projet 'Git 'local 'SaaS tenant 'automatisation locale.

PHASE 1 — FONDATIONS

Profil DevOps / Full Stack junior
Projet Kram Team Fitness Management
Kram Team

PROJET

KRAM TEAM — CONTEXTE

Kram Team Fitness Management Platform est une application de gestion de salle (kickboxing, MMA, fitness) : CRM adhérents, abonnements, planning, réservations, check-in QR, paiements, Green Rewards, dashboards multi-rôles.

STACK

- Backend : Laravel 12, PHP 8.3+, API REST, Sanctum — dossier backend/
- Frontend : React 19, TypeScript, Vite, Tailwind — dossier frontend/
- Données : PostgreSQL (cible) / MySQL possible en local selon config
- Cache & files : Redis + queues Laravel (quand activés)
- Local actuel : Laragon (Windows) — API :8001, app :5173

MODULES

- Salles / espaces clients (cible SaaS multi-tenant)
- Membres (CRM adhérents, QR, fiches)
- Abonnements & offres (quotas, statuts)
- Coachs (planning, taux horaire, rapports)
- Paiements & notifications
- Planning & réservations (validation staff)
- Check-in QR & présences
- Green Rewards (actions éco + boutique)

RÔLES

- Super Admin / Admin plateforme
- Propriétaire / gestionnaire de salle (reception)
- Coach
- Membre (adhérent)

À la fin de la Phase 1 : le stagiaire maîtrise le projet, a un repo Git propre, un environnement local reproductible, et une première logique SaaS (tenant) testable en local — sans déploiement VPS obligatoire dans cette phase.

MISSIONS

1 • COMPRENDRE LE PROJET

Priorité

- Étudier le fonctionnement global de la plateforme (parcours admin, réception, coach, membre).
- Comprendre les modules : salles, membres, abonnements, coaches, paiements, planning, réservations, QR, Green Rewards.
- Comprendre l'architecture Laravel + React + PostgreSQL (API 'front, modules backend/app/Modules).
- Comprendre le fonctionnement SaaS / multi-tenant (une plateforme, plusieurs salles / espaces clients).
- Identifier les rôles : Super Admin, propriétaire de salle, coach, membre...
- Faire un schéma de l'architecture actuelle (composants, flux de données, rôles).
- Documenter les dépendances et les prérequis (PHP, Node, DB, extensions, ports).

Livable : Document technique + schéma de l'application (docs/stage/ARCHITECTURE.md + schéma).

2 • GIT / GITHUB

- Comprendre le repository (structure backend/, frontend/, ANNEX/, docs/).
- Organiser les branches : main / develop / feature/* (convention d'équipe).
- Vérifier .gitignore et .env.example (aucun secret dans Git).
- Documenter la procédure de démarrage (clone 'install 'migrate 'serve).
- Travailler sur feature/devops-phase1-<prenom> ; commits clairs ; PR vers develop.

Livable : docs/stage/GIT_WORKFLOW.md + README démarrage à jour.

MISSIONS

3 • ENVIRONNEMENT LOCAL

- Installer / configurer Laravel, React, PostgreSQL et les dépendances.
- Préparer un environnement reproductible (étapes écrites + scripts si possible).
- Lancer API (port 8001) et front (port 5173) ; valider login démo.
- Commencer à utiliser Docker progressivement (compose pour DB/Redis, puis app).
- Noter les écarts Laragon vs Docker et les choix retenus.

Livable : docs/stage/LOCAL_SETUP.md + stack locale qui démarre en suivant la doc.

4 • PRÉPARATION SAAS

- Comprendre le concept de Tenant (espace client = une salle / une organisation).
- Définir comment isoler les données de chaque salle (tenant_id, scopes, policies).
- Cartographier les tables / modules qui doivent être multi-tenant.
- Préparer la logique de création d'un nouvel espace client (modèle + service).
- Documenter les risques (fuite de données entre tenants, migrations).

Livable : docs/stage/SAAS_TENANT_DESIGN.md (modèle d'isolement + plan technique).

MISSIONS & CLÔTURE

5 • AUTOMATISATION LOCALE

- Automatiser la création d'un nouveau tenant (commande Artisan ou script).
- Enchaîner : Nouveau client 'nouveau tenant 'configuration 'compte admin.
- Tester le scénario de bout en bout en local (données isolées, login admin salle).
- Documenter la commande / script et les paramètres (.env, seed).
- Démo courte au tuteur + PR Phase 1.

Livrable : Commande/script testé + docs/stage/TENANT_ONBOARDING.md + démo « nouveau client ».

RÈGLES

- Phase 1 = local + Git. Pas d'obligation de mise en prod VPS.
- Ne jamais committer .env, dumps, ni secrets.
- Documenter dans docs/stage/ (markdown + schéma).
- Chaque mission = commits + PR reviewable.
- Sécurité multi-tenant : aucune donnée d'une salle visible par une autre.

CRITÈRES DE FIN DE PHASE 1

- Schéma + doc technique validés par le tuteur.
- Workflow Git et démarrage local documentés.
- Environnement local reproductible (idéalement avec amorce Docker).
- Design tenant documenté.
- Création automatisée d'un tenant testée en local (nouveau client 'admin).

Scénario de validation : Nouveau client 'nouveau tenant 'configuration 'compte admin salle — testé et documenté en local.